UPC - UB - URV

MAI - Master's degree in Artificial Intelligence

EC Project:

Optimizing a difficult function with Evolution Strategies

Computational Intelligence

Participants:

Antonio Arcas Pons
Ricard Garcia Isern

Academic Year: 2025-26

Contents

1 Introduction **2**

2 Problem and Methodology **2**

2.1 Rastrigin Function 2

2.2 Evolution Strategy (ES) Implementation 3

2.3 Gradient Descent Methodology 4

3 Experimental Design & Results **5**

3.1 1st Experiment 6

3.1.1 Results Experiment 1 6

3.2 2nd Experiment 7

3.2.1 Results Experiment 2 7

3.3 3rd Experiment 8

3.3.1 Results Experiment 3 9

4 Discussion **9**

4.1 Experiment 1 9

4.2 Experiment 2 10

4.3 Experiment 3 10

5 Conclusion & Future Work **10**

1 Introduction

Traditional, derivative-based methods are efficient for simple, convex problems, but they suffer in complex, non-differentiable, and high multimodal search spaces. They are inherently local and are easily trapped in sub-optimal solutions.

This work employs Evolution Strategies (ES), a class of global optimization algorithms that are derivative-free and population-based. By leveraging stochastic mutation, with self-adaptation of strategy parameters, ES can effectively explore complex search spaces and avoid local traps.

We benchmark our ES implementation on the Rastrigin function. This function is characterized by having a huge number of deceptive local optima that surround the single global optimum at x=0.

The main objective is to implement a (μ, λ)-ES and analyze its performance. Based on 30 independent trials, we evaluate its convergence speed and its scalability as the problem dimension n increases.

2 Problem and Methodology

2.1 Rastrigin Function

The selected function for this study is the Rastrigin function [1]. Which is defined in n dimensions as:

$$f(\mathbf{x}) = An + \sum_{i=1}^{n} \left(x_i^2 - A\cos\left(2\pi x_i\right)\right)$$

Where A is a parameter that controls the amplitude of the sinusoidal component. For our experiments, we use the standard value A=10.

The Rastrigin function is highly multimodal, meaning its landscape is populated by a vast number of local optima. The $A\cos(2\pi x_i)$ term creates a high-frequency ripple across the entire search space, while the x_i^2 term creates an underlying parabolic bowl shape. This combination is deceptive: the local optima can easily trap local search algorithms, preventing them from finding the global solution, Figure 1.

Despite this complexity, the function has a single, known global optimum at the origin, $x = 0$ (where $x_i = 0$ for all i), at which point the function has a value of $f(0) = 0$.

For all experiments, the search domain was defined as the compact set $x \in [-2, 2]^n$. While the standard domain for this function is $x \in [-5.12, 5.12]^n$, we selected this smaller, more constrained search space. This decision serves a critical practical purpose: it significantly reduces the total search volume, which grows exponentially with dimension n. This reduction makes the optimization problem more computationally tractable. Importantly, this domain retains the function's essential difficulty: it is still highly multimodal and deceptive, and it contains the global optimum at $x = 0$ along with several surrounding local optima, allowing for a valid analysis of the ES's performance.

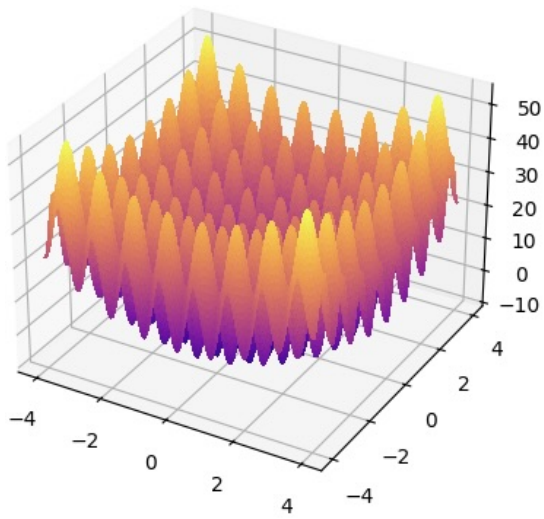

Figure 1: Visualization of the Rastrigin search space.

2.2 Evolution Strategy (ES) Implementation

To optimize the Rastrigin function, we implemented a (μ, λ)-Evolution Strategy. In this model, μ refers to the number of parents selected to generate λ number of children. These children then deterministically replace the entire previous generation. This is a non-elitist strategy that promotes exploration. The best solutions are not guaranteed to survive unless they are rediscovered by the offspring.

Each individual i in the population is represented both by $\mathbf{x}$ and σ. $\mathbf{x}$ is the candidate solution vector, representing a point in the n-dimensional search space (coordinates on the Rastrigin function). On the other hand, σ is a vector representing the strategy parameters (mutation step sizes), one for each dimension. Both $\mathbf{x}$ and $\sigma \in R^n$.

In this kind of algorithm, mutation is the only variation operator. We established a self-adaptative mutation with n uncorrelated step sizes, allowing the algorithm to learn the appropriate mutation strength for each dimension independently, being this the diagonal self-adaptive mutation.

The mutation process is applied to a copy of a parent to create a child. This process occurs in two different stages. First, the strategy parameter vector σ is mutated. This allows the step sizes to co-evolve with the solution. The update rule is the following one, which combines a global learning rate τ' with a dimension-specific learning rate τ. Each new σ_i' is calculated from the parent's σ_i as:

$$\sigma_i' = \sigma_i \cdot \exp\left(\tau' \cdot N(0,1) + \tau \cdot N_i(0,1)\right)$$

Here, $N(0,1)$ is a single random number drawn from a standard normal distribution and shared across all dimensions, while $N_i(0,1)$ is a new random number drawn for each dimension i. The learning rates are set based on problem dimension n as:

$$\tau = \frac{1}{\sqrt{2\sqrt{n}}} \qquad \text{and} \qquad \tau' = \frac{1}{\sqrt{2n}}$$

To prevent the search from stalling, a minimum floor value $\sigma_{\text{floor}} = 10^{-5}$ is enforced.

Second, the solution vector $\mathbf{x}$ is mutated using these newly updated strategy parameters σ'. This is achieved by adding Gaussian noise, where the magnitude of the noise in each dimension is scaled by its corresponding σ'_i:

$$x'_i = x_i + \sigma'_i \cdot N_i(0,1)$$

No recombination operator is used in our implementation; mutation is the only mechanism for generating variation. For selection, a deterministic (μ, λ) truncation is applied. At the end of each generation, the λ children are evaluated. The μ individuals with the best fitness (lowest Rastrigin score) are selected to become the parents for the next generation. All λ children from the current generation are then discarded, fulfilling the non-elitist replacement strategy.

2.3 Gradient Descent Methodology

For comparison against our derivative-free ES, we selected the L-BFGS-B [2] algorithm, a gradient-based optimization method. L-BFGS-B is a quasi-Newton algorithm that uses the function's first derivative to build an approximation of its second derivative. This algorithm was chosen to provide a direct comparison between our ES and a method that has full knowledge of the problem's gradient. As L-BFGS-B is a local optimizer by nature, it would fail on the multi-modal Rastrigin function if used alone. We therefore implemented it within a multi-start wrapper: for each of the 30 experimental runs, the L-BFGS-B algorithm was executed 1000 times from different random starting positions, and only the best result from all 1000 attempts is recorded. This transforms a method that detects a local optimum into a stochastic global optimizer.

3 Experimental Design & Results

Three different experiments were designed to get more information about our ES algorithm implementation in the context of our particular optimization problem. Each algorithm in this practical was run 30 times and metrics were averaged due to their stochastic nature. Additionally, throughout all experiments we fixed the following parameters:

- **Objective function to optimize:** Rastrigin
- **Stagnation window:** 10 generations
- **Stagnation threshold:** 10^{-6}
- **Maximum iterations:** 1000
- **Target fitness:** 0.01
- **Sigma floor (minimum value for self-adapting sigma):** 10^{-5}
- **Random seed:** 23
- **Runs per model:** 30
- **Dimensionality of the problem:** 4
- **Search bounds:** $[-2, 2]$ for all dimensions

Also note that the ES algorithm has three different stopping criteria:

- Max number of iterations (generations)
- Reaching a predefined performance (target fitness)
- Stagnation of fitness over generations

3.1 1st Experiment

We compare different parameter combinations of our ES algorithm. Different models are created and are faced against the optimization problem. The goal is to see if there exists at least an ES configuration that performs better than the others based on the calculated metrics. Specifically, we are testing how selection pressure and step size influence model performance. The recorded metrics (averaged over the 30 runs) are:

- Execution time
- Best fitness
- Number of fitness function calls
- Number of generations
- Success rate: out of the 30 runs, proportion that reached the target fitness stop.

To determine if the observed differences between configurations are statistically significant or merely due to chance, a Kruskal-Wallis H-test will be performed for each metric. If a significant difference is found, a Dunn's post-hoc test will be used to identify which specific pairs of configurations are different. On another note, the number of tested models is the same as the number of possible combinations of parameters to test. The parameters that are tested are the following:

- `step_sizes` = 0.1, 0.5, 1.0
- μ and λ pairs = (50, 1000), (50, 500), (50, 250)

3.1.1 Results Experiment 1

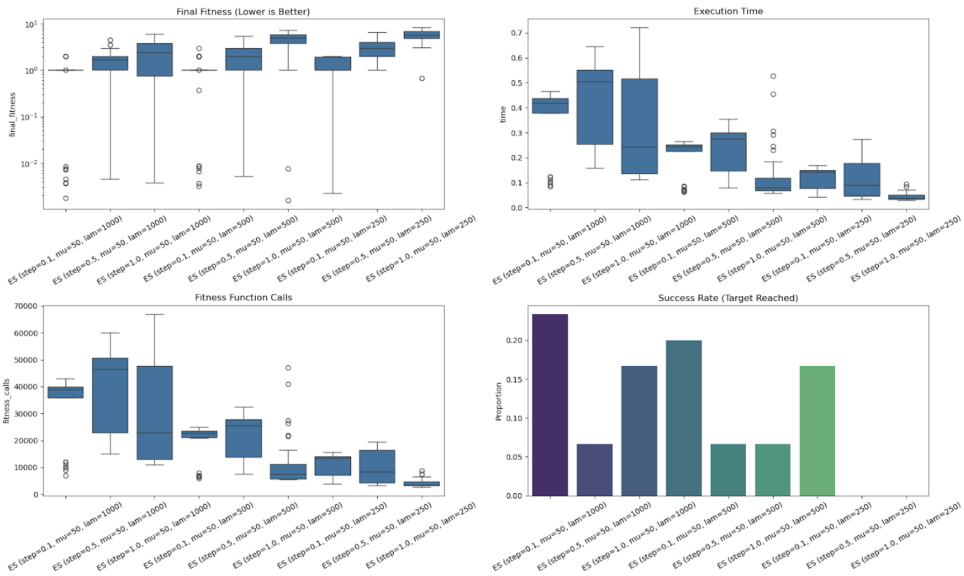

Figure 2: ES Parameter Tuning Results.

Based on success rate, the configuration `ES (step=0.1, mu=50, lam=1000)` was identified as the best-performing model.

With regard to the statistical analysis performed, a statistically significant difference was found among the nine configurations regarding final fitness. The best configuration performed significantly better than most configurations with larger step sizes and smaller μ/λ ratio. No significant difference was found between the top three configurations `ES (step=0.1, lam=1000)`, `ES (step=0.1, lam=500)` with p=1.0000, and `ES (step=0.1, lam=250)` with p=0.7298. Additionally significant differences were found in other metrics such as execution time and fitness calls (which of course are related) and number of generations (key factor was step size). For success rate, in our opinion the most interesting metric, the difference between configurations was significant. However, when applying the Post-Hoc test we did not find a significant difference between any pair of configurations.

3.2 2nd Experiment

The goal of this experiment is to compare the best-performing ES configuration (determined by Experiment 1) against a gradient based optimization algorithm. The competitor algorithm selected is L-BFGS-B, implemented within a multi-start wrapper. For each of the 30 runs, the algorithm was initialized from n different starting points, and the best result from all starts was recorded. The number of starts (n) was set to 1000, matching the offspring population size of the ES. This experiment was also conducted on the 4th-dimensional Rastrigin problem with search bounds of $[-2,2]$ for each dimension.

This experiment focuses on two metrics:

- Execution time
- Average optimization solution

We record the mean of the 30 final vectors to check how far this vector is from the true minimum at 0. We also measure the variance of the final solutions. To determine if the differences in these key metrics are statistically significant, the Mann-Whitney U test will be used.

3.2.1 Results Experiment 2

The L-BFGS-B algorithm achieved a 100% success rate, consistently converging to the true global minimum (average final fitness of 0.0000). The ES (Best Config) algorithm achieved a success rate of 23.33%, with an average final fitness of 0.8635.

In terms of computational cost, the L-BFGS-B algorithm was significantly faster, with an average execution time of 0.1723 seconds, compared to 0.3515 seconds for the ES. The L-BFGS-B algorithm also exhibited lower variance in its runtime (std 0.0164) compared to the ES (std 0.1409).

The Mann-Whitney U test demonstrated that L-BFGS-B was superior to our ES in all metrics (execution time, final fitness, solution accuracy) and that those differences were statistically significant.

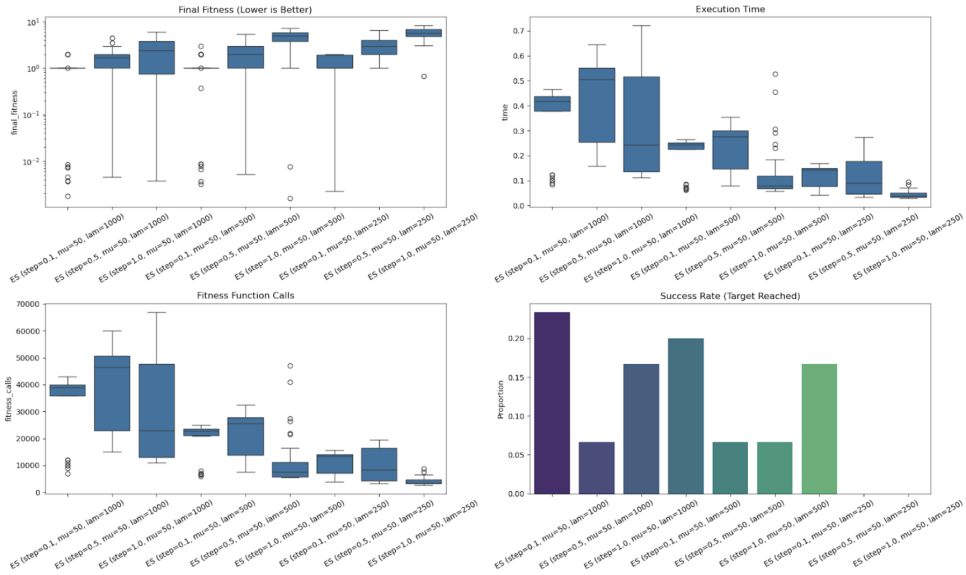

Figure 3: Algorithm Comparison Over 30 runs.

3.3 3rd Experiment

This experiment investigates the scalability of our best ES configuration. The goal is to determine how performance is affected by increasing dimensionality. The best ES configuration (identified in experiment 1) was run 30 times to optimize the rastrigin function with an increasing number of dimensions. The dimensions tested were: 2, 4, 8 and 16.

The search bounds were kept at $[-2, 2]$ for all dimensions. All metrics (execution time, best fitness, fitness calls, generations & success rate) were recorded to observe the scaling effect. Similar to Experiment 1, a statistical analysis was performed to see if the increase in dimensionality has a significant effect on performance. The Kruskal-Wallis H-test will be used to check for any significant difference across the four dimensional levels. If a difference is found, Dunn's post-hoc test will be used to know where the performance changes.

3.3.1 Results Experiment 3

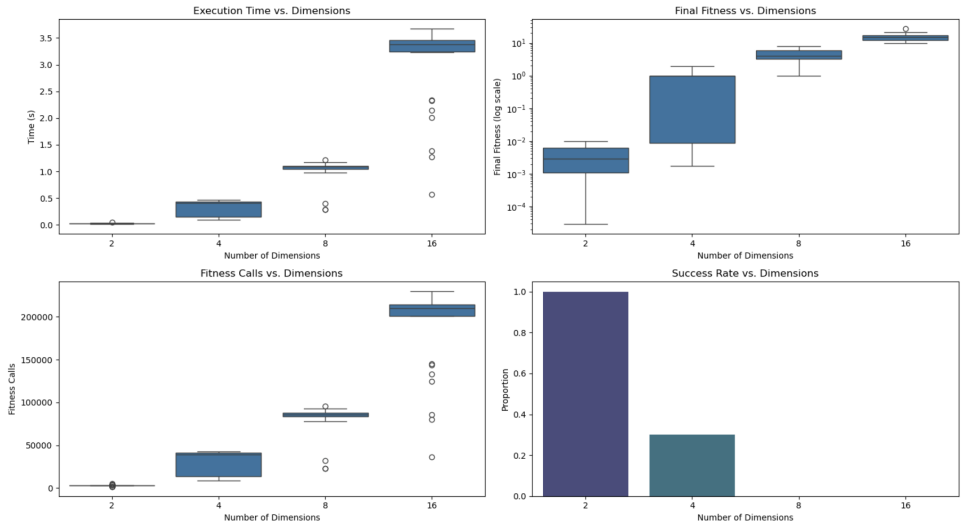

Figure 4: ES Scalability Analysis.

The results show a strong relationship between dimensionality and performance. As the number of dimensions increased, all performance metrics diminished. Average final fitness, execution time, fitness calls, and generations all increased with problem size. The algorithm's ability to find the target solution fell from a perfect 100% in the 2D case to 0% for all problems 8D and higher. These results were backed up statistically by the Kruskal-Wallis H-test. For final fitness, time, fitness calls and generation the difference between every level of dimensionality was statistically significant. For success rate, the difference between 2D, 4D and 8D is significant while the difference between 8D and 16D is not (since they share a success rate of 0).

4 Discussion

4.1 Experiment 1

The initial step size (`step_size`) was the most decisive factor for solution quality. All configurations with `step_size=0.1` (top performing ES) achieved a significantly better `final_fitness` than most configurations with larger step sizes. Larger steps probably "overshot" the global minimum, causing the algorithm to become permanently trapped in suboptimal local minima far from the origin.

Secondly, the selective pressure was the dominant factor for performance and efficiency. Our "winning" configuration (`step=0.1`, $\mu = 50$, $\lambda = 1000$) used the highest ratio of 20. This ratio not only provokes high selective pressure but also leads towards greater exploration because of the increase in the number of possible solutions (children) created. This is key to have more chances of finding the optimum. Conversely, the configurations with the lowest ratio of 5 (e.g., `step=1.0`, $\lambda = 250$) performed worst, failing in 100% of runs. These configurations, combined with a large step size, suffered from premature convergence (stagnation). Thus, running for fewer generations.

4.2 Experiment 2

L-BFGS-B was superior in every conceivable metric. This experiment shows the difference between a derivative-free optimizer and a gradient-based optimizer. ES, with no prior knowledge, must stochastically search for the minimum while L-BFGS-B, given the gradient, can efficiently descend to it. The ES's high bias (0.155) and spread (0.823) show that when it failed, it was consistently trapped in various local minima. L-BFGS-B's bias and spread of 0.0 prove that its multi-start wrapper was effective at finding the one true global minimum over all 30 runs.

These results prove that if a problem's gradient is known, a gradient-based method is the way to go. The ES would be the preferred algorithm for "true" black-box problems where a gradient is unavailable or non-existent. As referred in class, evolutionary algorithms are plan B.

4.3 Experiment 3

The results of experiment 3 show how our ES implementation fails to tackle high dimension problems. The reason for this failure lies in the implementation. We decided to use, uncorrelated mutation (one sigma per dimension: diagonal self-adaptive mutation). This mutation strategy cannot learn correlations between variables. As the search space grows exponentially, the probability of finding the narrow path to the global minimum by only mutating along the axes (x, y, z...) approaches zero. This explains why our 4D experiment (Exp 1 & 2) was so challenging, with only 23.3% success. A more advanced method, such as a CMA-ES (Covariance Matrix Adaptation Evolution Strategy), would be required to solve this problem at 8D or 16D, as it is specifically designed to learn correlations and "rotate" its search ellipse to navigate high-dimensional, non-convex landscapes.

5 Conclusion & Future Work

In this practical we implemented and benchmarked a (μ,λ)-Evolution Strategy algorithm. Our three experiments gave us more information about our algorithm's capabilities and limitations. The parameter tuning (Experiment 1) demonstrated that a small initial step size (0.1) and a high exploration ratio ($\lambda/\mu = 20$) were optimal for the 4D Rastrigin problem, achieving a 23.3% success rate. However, this "best" configuration was outperformed in every metric by a classic gradient-based L-BFGS-B algorithm, which consistently found the true global optimum (Experiment 2). The final scalability analysis (Experiment 3) revealed that the algorithm's simple, uncorrelated mutation strategy is insufficient to overcome the curse of dimensionality. While the ES solved the 2D problem, its performance collapsed at 8D and higher. We conclude that while our ES is a functional black-box optimizer, the chosen mutation strategy makes it non-scalable and therefore not suited for this class of complex, high-dimensional problems.

References

[1] Wikipedia contributors. Rastrigin function, 2025. Last edited 20 April 2025.

[2] Antoine Guitton. The l-bfgs-b algorithm. Webpage, part of Stanford Exploration Project Report 117, Oct 2004. Accessed on [Your Access Date].